32

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

유닛 테스트와 테스트 더블

제2부 NestJS 핵심 · 8장 테스트 기초

내 마음대로 안 되는 것에 기대는 코드를 빠르고 매번 같게 시험하는 방법



© 한상훈 NestJS 백엔드 마스터

빨간 테스트 세 개

○ ○ ○ 터미널

```
$ npx vitest run
 × reservations.service.spec.ts > 결제사를 실제로 부르지 않고 예매를 시험한다 3ms
 × notifier.controller.spec.ts > NotifierController > root > should return "Hello World!" 216ms
 × app.controller.spec.ts > AppController > root > should return "Hello World!" 217ms

AssertionError: expected '예매 API: A구역 3열 15번' to be 'Hello World!'
AssertionError: expected '알림: A구역 3열 15번 예매 확정' to be 'Hello World!'
TypeError: Cannot read properties of undefined (reading 'setTag')

 Test Files  3 failed | 3 passed (6)
      Tests  3 failed | 3 passed (6)
```

© 한상훈 NestJS 백엔드 마스터

낡은 테스트

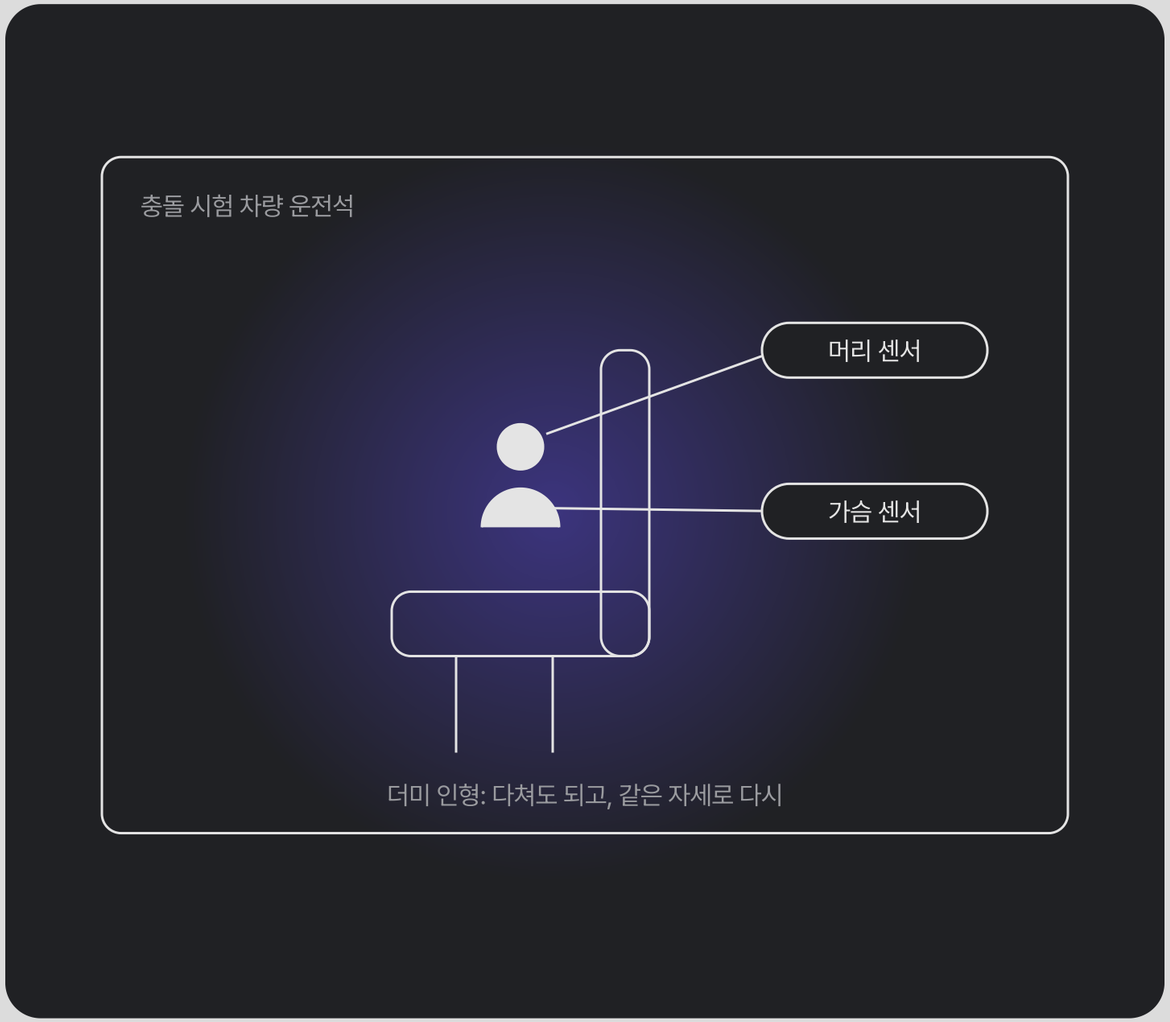
테스트	실패 이유	고친 방법
Hello World 두 개	첫 화면 문구는 바뀌고 기댓값은 그대로	기댓값과 테스트 이름을 지금 문구로
예매 서비스	생성자 인자 1개 → 3개, 테스트의 new는 그대로	새로 작성

→ 빨간 테스트가 늘 섞여 있으면: 새로 깨진 테스트가 그 사이에 묻힘

→ 실패는 그날 수정

© 한상훈 NestJS 백엔드 마스터

충돌 시험의 더미 인형



→ 사람 대신 인형: 다쳐도 되고, 같은 자세로 몇 번이든 다시

→ 인형의 센서: 머리에 간 힘을 사람처럼 숫자로 기록

→ 테스트 더블(test double): 결제사, 문자 발송, 시계 자리에 앉히는 대역

© 한상훈 NestJS 백엔드 마스터

유닛 테스트

- → 유닛 테스트(unit test): 클래스나 함수 하나를 떼어 내서 하는 시험
- → 떼어 내기: 그 클래스가 기대는 다른 클래스는 테스트 더블로 교체
- → 네트워크, DB, Nest 부팅 없음
- → 밀리초 단위, 몇 번을 돌려도 같은 결과
- → 파일: 시험할 파일 옆의 `*.spec.ts`

© 한상훈 NestJS 백엔드 마스터

좌석 서비스 첫 테스트

```ts
// seats/seats.service.spec.ts
describe('SeatsService', () => {
  let service: SeatsService;

  beforeEach(() => {
    const performances = {
      findOne: (id: number) => ({ id, concertId: 1, startsAt: '2026-12-24T19:00' }),
    } as unknown as PerformancesService;
    service = new SeatsService(performances);
    service.open(1);
  });

  it('빈 좌석을 붙잡으면 HELD가 된다', () => {
    const seat = service.hold(1, 'A-3-15');

    expect(seat.status).toBe('HELD');
  });

  it('이미 붙잡힌 좌석을 또 붙잡으면 409', () => {
    service.hold(1, 'A-3-15');

    expect(() => service.hold(1, 'A-3-15')).toThrow(SeatUnavailableError);
  });
});
```

→ `describe`: 테스트 묶음 / `it`: 테스트 하나 / `expect`: 확인

→ `SeatUnavailableError`: 좌석 서비스가 던지는 좌석 충돌 에러, 응답으로는 409

→ 준비(Arrange), 실행(Act), 확인(Assert)

→ `beforeEach`: it마다 새 좌석 서비스, 테스트끼리 결과가 섞이지 않게

© 한상훈 NestJS 백엔드 마스터

첫 실행

터미널

```
$ npx vitest run apps/ticketing/src/seats
 Test Files  1 passed (1)
      Tests  3 passed (3)
   Duration  440ms (transform 65ms, setup 0ms, import 290ms, tests 3ms, environment 0ms)
```

→ 서버도, Nest도 띄우지 않음

→ 회차 서비스 자리: `findOne` 하나만 있는 객체

데이터 블록: vitest 4.1.12(Nest CLI 12 새 프로젝트 기본), 강사 PC 실측 Node v24.11.0, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

스텁

→ 스텁(stub): 미리 정해 둔 답만 돌려주는 더블

→ 질문이 오면 적어 둔 답을 그대로

→ 쓰는 곳: 만들기 번거로운 상황을 한 줄로

→ 예: 회차가 있는 상황, 회차가 없는 상황

→ 스텁으로 보는 것: 회차 서비스의 답에 따른 좌석 서비스의 판단

© 한상훈 NestJS 백엔드 마스터

예외를 던지는 스텁

```
  it('없는 회차에는 좌석을 열 수 없다', () => {
    const performances = {
      findOne: (id: number) => {
        throw new NotFoundException(`회차 ${id}번이 없습니다`);
      },
    } as unknown as PerformancesService;
    const service = new SeatsService(performances);

    expect(() => service.open(9)).toThrow('회차 9번이 없습니다');
  });
```

→ 진짜 회차 서비스가 없는 회차에 하는 일과 같은 모양: NotFoundException

© 한상훈 NestJS 백엔드 마스터

진짜 회차 서비스를 쓰면

	진짜 회차 서비스	스텁
회차가 있는 상황	공연 등록 → 회차 등록 → 좌석 열기	`findOne` 한 줄
회차 쪽에 버그	좌석 테스트도 실패	좌석 테스트는 그대로
실패를 봤을 때	좌석인지 회차인지 다시 확인	좌석 서비스 문제
회차에 DB가 붙으면	좌석 테스트에도 DB 필요	그대로

→ 스텁의 모양은 진짜와 같게: 진짜가 예외를 던지면 스텁도 예외

© 한상훈 NestJS 백엔드 마스터

예매 서비스 테스트 준비

```
describe('ReservationsService', () => {
  const gateway = { charge: vi.fn(), refund: vi.fn() };
  const logger = { setTag: () => {}, log: () => {} };
  const notifications = { sendConfirmed: vi.fn() };
  let service: ReservationsService;

  beforeEach(() => {
    vi.resetAllMocks();
    gateway.charge.mockResolvedValue({ approvalId: 'pg-110000', amount: 110000 });
    notifications.sendConfirmed.mockReturnValue('[문자 → 김민수] A-3-15 예매가 확정되었습니다');
    service = new ReservationsService(
      gateway as unknown as PaymentGateway,
      logger as unknown as TaggedLogger,
      notifications as unknown as NotificationsService,
    );
  });
```

→ `vi.fn()`: 불릴 때마다 받은 인자를 기록하는 가짜 함수

→ `vi.resetAllMocks()`: 테스트마다 기록 비우기

© 한상훈 NestJS 백엔드 마스터

목으로 확인하기

```
  it('결제가 되면 확정 문자를 한 번 보낸다', async () => {
    const result = await service.reserve('A-3-15', 110000, '김민수');

    expect(result.approvalId).toBe('pg-110000');
    expect(notifications.sendConfirmed).toHaveBeenCalledTimes(1);
    expect(notifications.sendConfirmed).toHaveBeenCalledWith('sms', '김민수', 'A-3-15');
  });

  it('결제가 실패하면 확정 문자를 보내지 않는다', async () => {
    gateway.charge.mockRejectedValue(new Error('카드 한도 초과'));

    await expect(service.reserve('A-3-15', 110000, '김민수')).rejects.toThrow('카드 한도 초과');
    expect(notifications.sendConfirmed).not.toHaveBeenCalled();
  });
```

© 한상훈 NestJS 백엔드 마스터

문자를 먼저 보내는 코드

```
  async reserve(seatId: string, amount: number, buyer: string, channel = 'sms', userAgent = 'unknown') {
    const message = this.notifications.sendConfirmed(channel, buyer, seatId);  // 위로 올림
    const payment = await this.gateway.charge(amount);
```

○ ○ ○ 터미널

```
✓ ReservationsService > 결제가 되면 확정 문자를 한 번 보낸다 3ms
× ReservationsService > 결제가 실패하면 확정 문자를 보내지 않는다 5ms
  → expected "vi.fn()" to not be called at all, but actually been called 1 times
 1st vi.fn() call:
   Array [ "sms", "김민수", "A-3-15" ]
```

→ 결제 성공 테스트: 차이 없음

→ 결제 실패 테스트: 확정 문자 발송을 바로 잡아냄

© 한상훈 NestJS 백엔드 마스터

목과 더미

→ 목(mock): 불린 기록을 남기고, 테스트가 그 기록을 확인하는 더블

→ 확인하는 것: 몇 번, 무슨 인자로, 한 번도 안 불렸는지

→ 쓰는 곳: 반환값에 안 보이는 일, 부수 효과 (문자 발송, 메일, 결제, 이벤트 발행)

→ 더미(dummy): 생성자 자리만 채우는 더블, 테스트가 아무것도 확인하지 않음 (로거)

© 한상훈 NestJS 백엔드 마스터

페이크 결제사와 스파이

```
it('페이크 결제사로 예매하면 fake 승인 번호가 붙는다', async () => {
  const gateway = new FakePaymentGateway();
  const charge = vi.spyOn(gateway, 'charge');
  const service = new ReservationsService(
    gateway,
    { setTag: () => {}, log: () => {} } as unknown as TaggedLogger,
    { sendConfirmed: () => '' } as unknown as NotificationsService,
  );

  const result = await service.reserve('A-3-15', 110000, '김민수');

  expect(result.approvalId).toBe('fake-110000');
  expect(charge).toHaveBeenCalledWith(110000);
});
```

→ 페이크(fake): 진짜처럼 동작하지만 운영에는 못 쓸 만큼 가볍게 만든 구현

→ 스파이(spy): 진짜 메서드는 그대로 돌리고 호출만 기록

© 한상훈 NestJS 백엔드 마스터

다가오는 회차 테스트

```
beforeEach(() => {
  const concerts = { findOne: (id: number) => ({ id }) } as unknown as ConcertsService;
  service = new PerformancesService(concerts);
  service.create({ concertId: 1, startsAt: '2026-10-01T19:00' });
  service.create({ concertId: 1, startsAt: '2026-12-24T19:00' });
});

it('지금보다 뒤에 시작하는 회차만 나온다', () => {
  expect(service.findUpcoming().map((p) => p.startsAt)).toEqual(['2026-12-24T19:00']);
});
```

→ findUpcoming: 안에서 new Date()로 지금 시각을 읽음 → 오늘 실행: 통과

© 한상훈 NestJS 백엔드 마스터

12월 25일의 실패

터미널

```
 × PerformancesService > 지금보다 뒤에 시작하는 회차만 나온다 8ms
   → expected [] to deeply equal [ '2026-12-24T19:00' ]

- Expected
+ Received

- [
-   "2026-12-24T19:00",
- ]
+ []
```

→ 시계를 2026-12-25로 옮겨서 같은 테스트 실행

→ 코드 변경 없음, 날짜만 바뀌어 실패

© 한상훈 NestJS 백엔드 마스터

시계 고정

```
beforeEach(() => {
  vi.useFakeTimers({ toFake: ['Date'] });
  vi.setSystemTime(new Date('2026-11-01T00:00:00Z'));
  // 회차 두 개 등록
});

afterEach(() => {
  vi.useRealTimers();
});

it('시작 시각이 지난 회차는 빠진다', () => {
  vi.setSystemTime(new Date('2026-12-25T00:00:00Z'));

  expect(service.findUpcoming()).toEqual([]);
});
```

→ 가짜 타이머: 테스트 안의 지금을 고정, Date만 교체

→ 끝나면 진짜 시계로 복구

→ 다른 방법: 지금 시각을 주는 시계 객체를 생성자로 받기

→ 예: 붙잡은 좌석을 10분 뒤 푸는 기능, 시계만 10분 뒤로

© 한상훈 NestJS 백엔드 마스터

진짜 결제사 클라이언트로 결제

```
it('진짜 결제사 클라이언트로 결제한다', async () => {
  const gateway = new HttpPaymentGateway(
    { mode: 'http', baseUrl: 'http://localhost:5032', timeoutMs: 3000 },
    'tok-123',
  );

  const payment = await gateway.charge(110000);

  expect(payment.approvalId).toBe('pg-110000');
});
```

○ ○ ○ 터미널

```
결제사 정상
 ✓ 진짜 결제사 클라이언트로 결제한다 43ms

결제사 무응답 (MODE=hang)
 × 진짜 결제사 클라이언트로 결제한다 3017ms
   → The operation was aborted due to timeout
```

→ 5032: 로컬 가짜 결제사 서버

→ 코드 변경 없음, 상대 서버 상태만 다름

© 한상훈 NestJS 백엔드 마스터

플래키 테스트

→ 플래키 테스트(flaky test): 같은 코드인데 실행마다 결과가 다른 테스트

→ 원인: 남의 서버, 네트워크, 시계, 테스트 순서

→ 결과: 실패하면 일단 재실행하는 습관 → 진짜 버그도 재실행으로 통과시킴

→ 예: 결제사 호출 테스트 200개 × 약 40ms = 8초, 무응답인 날 200개 × 3초 = 10분

→ 페이크를 넣은 같은 테스트: 1~2ms, 결제사 상태와 상관없이 같은 결과

→ 결제사와 맞는지 확인: 결제사 클라이언트만 따로 붙여 보는 테스트로 분리

© 한상훈 NestJS 백엔드 마스터

테스트 더블 다섯 가지

이름	하는 일	이 강의 예
더미	자리만 채움	예매 서비스의 로거
스텁	정해 둔 답을 돌려줌	좌석 서비스의 회차 서비스
스파이	진짜를 돌리면서 호출 기록	`vi.spyOn(gateway, 'charge')`
목	호출 기록을 테스트가 확인	확정 문자 `sendConfirmed`
페이크	가볍게 진짜처럼 동작	`FakePaymentGateway`, 가짜 타이머

→ 실무에서는 전부 묶어 목이라고 부르는 경우가 많음 (`vi.fn`, `vi.mock`)

© 한상훈 NestJS 백엔드 마스터

무엇을 바꾸나

- → 바꾸는 것: 내 코드 밖 (결제사, 문자 발송), 느리거나 매번 다른 것 (시계, 난수)
- → 바꾸지 않는 것: 시험할 클래스 자신
- → 더블의 한계: 진짜는 이렇게 동작할 것이라는 우리 믿음대로만 움직임
- → 진짜와 다르게 만든 더블: 테스트는 통과, 운영은 실패
- → 그래서 진짜와 붙여 보는 테스트가 따로 필요
- → 테스트 이름: 규칙 문장 그대로, 실패 목록이 깨진 규칙 목록

© 한상훈 NestJS 백엔드 마스터

전체 실행

터미널

```
$ npx vitest run
 ✓ performances.service.spec.ts > 지금보다 뒤에 시작하는 회차만 나온다 5ms
 ✓ performances.service.spec.ts > 시작 시각이 지난 회차는 빠진다 2ms
 ✓ seats.service.spec.ts > 빈 좌석을 붙잡으면 HELD가 된다 2ms
 ✓ seats.service.spec.ts > 이미 붙잡힌 좌석을 또 붙잡으면 409 1ms
 ✓ seats.service.spec.ts > 없는 회차에는 좌석을 열 수 없다 1ms
 ✓ reservations.service.spec.ts > 결제가 되면 확정 문자를 한 번 보낸다 2ms
 ✓ reservations.service.spec.ts > 결제가 실패하면 확정 문자를 보내지 않는다 1ms
 ✓ reservations.service.spec.ts > 페이크 결제사로 예매하면 fake 승인 번호가 붙는다 2ms
 ✓ app.controller.spec.ts > 첫 화면 문구에 좌석 표기가 들어간다 214ms
 ✓ notifier.controller.spec.ts > 첫 화면 문구에 좌석 표기가 들어간다 199ms
 (… CLI가 만든 should be defined 3개)
 Test Files  8 passed (8)
      Tests  13 passed (13)
```

데이터 블록: 전체 유닛 테스트 8 files, 13 passed, Duration 1.13s(강사 PC 실측 2026-10-26), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

유닛 테스트의 규칙

"유닛 테스트는 시험할 클래스 하나만 진짜로 두고,
나머지는 테스트 더블로 바꿔서 빠르고 매번 같게 돌립니다."